Adaptive Preconditioned Dynamics, Solution Geometry, and Generalization in Neural Networks

A Matched-Seed Study of SGD and Adam Across Parameter, Basin, Function, and Representation Geometry

Changyuan Yu(cy2812) Xiangyu Liao(xl3581)
Zhiliang Wang(zw3162) Kevin Sun(ys3978)

May 2026

Abstract

Adaptive optimizers are usually thought of as a way to speed up training, but in over-parameterized networks they also help *select* which low-loss solution training ends up at. To probe that selection effect, we compare SGD with momentum against Adam under a matched-seed protocol on Fashion-MNIST. The mathematical contrast is concrete: SGD applies a scalar spatial preconditioner, while Adam applies a time-varying diagonal one, and this is what reshapes the local error dynamics. Our analysis is organized around three project questions: *convergence*, *solution geometry*, and *generalization*. For solution geometry we test the optimizer-induced effect with six complementary measurements (distance from initialization, trajectory path length and update norm, Hessian sharpness/trace, perturbation flatness, inter-seed dispersion, and a loss-matched control), so that no single proxy carries the conclusion. For generalization we follow the resulting parameter-space difference through three additional levels of geometry: basin connectivity (linear mode connectivity), function-space agreement (prediction disagreement, symmetric KL, logit cosine), and representation alignment (linear CKA). Adam moves substantially farther from the starting point and selects more dispersed solutions, and on the MLP it lands in distinctly flatter regions even after loss matching. Despite this, the differences shrink as the comparison propagates: matched-seed cross-optimizer basin barriers are an order of magnitude smaller than same-optimizer cross-seed barriers, prediction disagreement and CKA fall within the same-optimizer cross-seed baseline, and only logit-direction cosine still shows a sharp cross-optimizer signal. The implicit bias of Adam vs. SGD is therefore real at the parameter level but is filtered through architecture, basin connectivity, and representation alignment before it reaches generalization.

1 Introduction

Modern neural networks have so many parameters that they can fit the training data in many different ways, and lots of those fits can get very close to perfect training accuracy. When that happens, the optimizer isn't just driving training error down — it is also picking *which* of those many near-perfect solutions training actually lands at. This selection effect is usually called the algorithm's "implicit bias" or "implicit regularization," and it can have a real impact on how the trained network behaves in practice.

So, do adaptive optimizers change that implicit bias? We focus on the comparison between SGD with momentum and Adam. SGD applies one global learning-rate scale to every coordinate of the stochastic gradient, while Adam rescales each coordinate using running estimates of the first

and second moments of recent gradients [1]. This coordinate-wise adaptation often helps in early optimization, especially when gradients vary strongly across layers, but it may also change the final solution and therefore generalization. Comparing the two head-to-head is what lets us see whether that's actually happening.

The overall project question is:

Do adaptive optimizers alter the implicit bias and generalization of neural networks?

We organize the study around three linked sub-questions:

1. **Early-stage convergence.** Does Adam converge faster than SGD at the beginning of training?

2. **Solution geometry.** If Adam is faster, does it reach a different kind of solution, or only the same solution earlier?

3. **Generalization.** Do the observed geometric differences explain differences in test performance and train–test gap?

These three questions form the spine of the report. Crucially, the geometry question is not tested with a single proxy: we use six complementary measurements (distance from initialization, cumulative path length and per-step update norm, Hessian sharpness/trace, isotropic perturbation flatness, inter-seed parameter dispersion, and a loss-matched control), so that no single quantity carries the verdict. The generalization question is then followed through three further levels of geometry that probe how a parameter-space difference can, or cannot, become a generalization difference: basin connectivity along the linear interpolant between two trained solutions, function-space agreement on test predictions, and representation alignment in penultimate-layer features.

The contributions of the report are accordingly:

1. We formulate SGD and Adam as two preconditioned stochastic dynamics and develop a local quadratic analysis $e_{t+1} \approx (I - P_t H)e_t$ that explains why a time-varying diagonal preconditioner can change the selected low-loss solution.

2. We use matched seeds (shared initialization, shared minibatch order) to isolate optimizer-induced effects from initialization and batch-order randomness, and add a same-optimizer cross-seed baseline as a falsification test.

3. We use the six geometry diagnostics above as a multi-faceted answer to the geometry question, so the verdict does not depend on a single proxy.

4. We carry the parameter-space difference through three further levels — mode connectivity for basin geometry; prediction disagreement, symmetric KL, and logit cosine for function geometry; feature-kernel alignment and linear CKA for representation geometry — and pull them together into a three-level synthesis: parameter, basin, and function–representation.

The answer isn't simply "Adam is better" or "SGD is better" — the experiments show a conditional pattern. On the MLP, Adam has a clear early-optimization advantage and lands in much flatter solutions according to the Hessian proxies (about $5\times$ lower $\lambda_{\max}$, $6\times$ lower trace), and this flatness gap survives the loss-matched control. Yet test accuracy and train–test gap differ by less than the seed-to-seed variation, so a flat-minima story alone doesn't explain generalization here. On the CNN, Adam is still slightly faster in training loss, but its dominant Hessian eigenvalue is actually *higher* than SGD's, and both optimizers reach similar test accuracy and gap. In other

words, adaptive optimization changes the training trajectory and the selected parameter region, but whether that matters for generalization depends on architecture, on whether the two endpoints share a basin, and on how much of the parameter difference survives at the predictor and representation levels.

1.1 Scope of Claims

The implicit-bias and generalization claims in this report are claims about Adam-vs-SGD trajectory and solution geometry *at the scale and protocol of this study*: small image-classification architectures on Fashion-MNIST, matched-seed protocol, no explicit regularization, tens of epochs, a handful of seeds. The MLP/SmallCNN pair is a controlled scale-down designed to expose mechanism, not a representative sample of modern deep learning. We do not claim the same effects, in the same direction or magnitude, transfer to large-scale settings like ResNets on ImageNet, transformer language models, or training horizons of 10^4+ steps. The full experimental protocol is in Section 3, and a list of caveats appears in Section 7.2.

2 Mathematical Background and Problem Description

Let $\widehat{L}(\theta)$ denote the empirical cross-entropy loss and let

$$g_t = \nabla_\theta \widehat{L}_{B_t}(\theta_t)$$

be the stochastic gradient on minibatch B_t. Rather than treating SGD and Adam as two unrelated engineering rules, we view both as preconditioned stochastic dynamics,

$$\theta_{t+1} = \theta_t - P_t q_t,$$

where q_t is a gradient-like direction and P_t is the effective preconditioner. For SGD with momentum,

$$u_t = \mu u_{t-1} + g_t, \qquad \theta_{t+1} = \theta_t - \eta u_t,$$

so momentum smooths gradients over time, while the spatial scaling remains scalar: $P_t = \eta I$. Expanding the recursion,

$$u_t = \sum_{k=0}^{t} \mu^{t-k} g_k,$$

shows that momentum is a temporal averaging mechanism, not a coordinate-wise metric change.

Adam instead computes

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^{\odot 2},$$

with bias-corrected moments

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}.$$

Its update can be written as

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} = \theta_t - \alpha D_t \hat{m}_t,$$

where

$$D_t = \operatorname{diag}\left(\frac{1}{\sqrt{\hat{v}_{t,i}} + \varepsilon} \right)_i.$$

Thus the key mathematical contrast is scalar spatial preconditioning for SGD versus time-varying diagonal preconditioning for Adam.

This distinction matters in local geometry. Near a low-loss point $\theta^\star$, approximate

$$\widehat{L}(\theta) \approx \widehat{L}(\theta^\star) + \tfrac{1}{2}(\theta - \theta^\star)^\top H(\theta - \theta^\star).$$

For $e_t = \theta_t - \theta^\star$, preconditioned gradient dynamics obey

$$e_{t+1} \approx (I - P_t H)e_t.$$

For SGD, this becomes $e_{t+1} \approx (I - \eta H)e_t$. If $H = Q\Lambda Q^\top$ and $\tilde{e}_t = Q^\top e_t$, then

$$\tilde{e}_{t+1,i} = (1 - \eta\lambda_i)\tilde{e}_{t,i}.$$

A single scalar learning rate controls all Hessian eigendirections. For Adam, however,

$$e_{t+1} \approx (I - \alpha D_t H)e_t,$$

and D_t generally does not commute with H. In the Hessian eigenbasis,

$$\tilde{e}_{t+1} = \big(I - \alpha Q^\top D_t Q\Lambda\big)\tilde{e}_t,$$

which can mix eigendirections. Equivalently, Adam responds to an effective preconditioned curvature

$$H_t^{\text{eff}} = D_t^{1/2} H D_t^{1/2}.$$

This is the central reason Adam should not be interpreted merely as a faster clock for SGD.

In an over-parameterized network, the low-loss set

$$\mathcal{S}_\varepsilon = \{\theta : \widehat{L}(\theta) \le \varepsilon\}$$

contains many parameter vectors. The optimizer acts as a selection map

$$\theta^\star_{\mathcal{A}} = A_{\mathcal{A}}(\theta_0, \{B_t\}, \text{hyperparameters}).$$

Our matched-seed protocol fixes both θ_0 and the minibatch sequence across the SGD and Adam runs, so any observed differences are more directly attributable to the update rule itself.

We use Hessian quantities as local geometry diagnostics. The dominant curvature is

$$\lambda_{\max}(H) = \max_{\|u\|_2=1} u^\top H u,$$

and the aggregate curvature is $\operatorname{tr}(H) = \sum_i \lambda_i(H)$. For a small perturbation,

$$\Delta\widehat{L}(\delta) \approx \tfrac{1}{2}\delta^\top H \delta.$$

If $\delta \sim \mathcal{N}(0, \sigma^2 I)$, then

$$\mathbb{E}[\Delta\widehat{L}(\delta)] \approx \frac{\sigma^2}{2}\operatorname{tr}(H).$$

This identity motivates the perturbation-flatness measurement that complements the Hessian proxies in Section 5.5.

Sharpness must be interpreted carefully. Euclidean Hessian spectra are parameter-space diagnostics, not invariant measures of function complexity: with ReLU networks, rescaling one layer and inversely rescaling the next can leave the represented function unchanged while changing parameter norms and curvature. We therefore use Hessian and perturbation metrics as controlled *within-protocol* diagnostics, and later test whether parameter differences survive in basin, function, and representation space.

2.1 Project Hypotheses

We test three linked hypotheses. First, Adam should reduce training loss faster than default SGD, because diagonal preconditioning adapts the step scale coordinate by coordinate. Second, Adam and SGD should select different parameter-space regions even under matched initialization and minibatch order, and this difference should show up on several complementary geometry diagnostics, not just one. Third, parameter-space geometry alone should not fully determine generalization: the parameter-level difference is filtered, in turn, through architecture, basin connectivity, function-space behavior, and representation similarity.

3 Experimental Setup

All experiments use Fashion-MNIST with the standard train/test split and normalized grayscale inputs. We compare two architectures: a batch-normalized MLP and a SmallCNN with two convolutional blocks followed by a small classifier. The MLP has weaker image-specific inductive bias, so optimizer-induced effects should be easier to observe; the CNN adds convolution, pooling, and weight sharing, allowing us to test whether architecture compresses optimizer differences.

The main experiments use five seeds: $\{42, 123, 456, 789, 2024\}$. For each seed, the SGD and Adam runs share the same initialization and the same deterministic data-loader shuffle. We deliberately use no weight decay, dropout, or data augmentation, so that explicit regularization does not obscure optimizer-induced effects.

Component	**Setting**
Dataset	Fashion-MNIST
Batch size	128
Epochs	30
Loss	Cross-entropy
Seeds	42, 123, 456, 789, 2024
SGD	learning rate 0.01, momentum 0.9, weight decay 0
Adam	learning rate 0.001, betas $(0.9, 0.999)$, weight decay 0
Architectures	Batch-normalized MLP and SmallCNN

Table 1: Main experimental protocol.

The measured quantities are organized by which of the three project questions they answer. *Convergence* (Part I): training-loss curves and threshold crossing. *Solution geometry* (Part II): six complementary measurements — distance from initialization, cumulative path length and per-step update norm, Hessian sharpness/trace, isotropic perturbation flatness, inter-seed parameter dispersion, and a loss-matched checkpoint comparison. *Generalization* (Part III): test accuracy and train–test gap, plus three further levels of geometry (linear mode connectivity, function-space disagreement, representation-space CKA) that test how a parameter-space difference propagates to predictions.

4 Part I: Early-Stage Convergence

Adam's most immediate expected advantage is faster early training, so we record the first global step at which each run crosses fixed training-loss thresholds. This part is mainly motivational: the

deeper question is not whether Adam reaches low loss earlier, but whether it selects a different kind of solution once it gets there.

Model	Threshold	SGD steps	SGD std	Adam steps	Adam std
MLP	2.0	5.4	0.5	2.6	0.5
MLP	1.0	12.8	1.9	6.8	0.8
MLP	0.5	54.2	1.3	30.8	4.6
MLP	0.2	643.2	137.7	477.0	96.7
SmallCNN	2.0	4.4	0.5	3.8	0.8
SmallCNN	1.0	9.4	2.1	8.8	1.5
SmallCNN	0.5	36.8	9.4	32.2	5.1
SmallCNN	0.2	346.8	38.9	312.2	49.6

Table 2: Mean global steps to first reach each training-loss threshold across five seeds. Lower is faster.

Adam reaches every threshold earlier on average. The effect is strongest on the MLP: at threshold 0.2, Adam needs 477.0 steps versus SGD's 643.2. On the CNN, Adam is still faster but the gap is smaller because both optimizers fit the architecture well. A small learning-rate sweep on the MLP shows that tuned SGD can narrow part of this early gap, so the defensible claim is the narrower one: Adam is faster than the default SGD baseline under the matched protocol, especially on the MLP. This motivates the next question — does Adam merely arrive at low loss earlier, or arrive at a different kind of solution?

5 Part II: Solution Geometry — Multi-Faceted Evidence

5.1 Question and Roadmap

The second project question asks whether Adam and SGD converge to geometrically different solutions, or whether Adam simply arrives at the same kind of solution faster. We answer this with six complementary measurements, organized so that each addresses a distinct way the question could be wrong:

1. Distance from initialization (Section 5.2) measures the net displacement of each optimizer from θ_0.
2. Trajectory decomposition (Section 5.3) splits that displacement into cumulative path length, directness ratio, and per-step update norm, distinguishing "Adam moves farther because it takes larger steps" from "Adam follows a more directed drift."
3. Hessian curvature (Section 5.4) probes the shape of the loss surface around each endpoint via $\lambda_{\max}(H)$ and $\mathrm{tr}(H)$.
4. Perturbation flatness (Section 5.5) measures the Hessian proxies operationally by recording how much the loss increases under controlled isotropic perturbations.
5. Inter-seed parameter dispersion (Section 5.6) checks whether an optimizer reproducibly lands in similar regions, or scatters across the parameter space.

6. Loss-matched control (Section 5.7) rules out the possibility that the geometric differences are simply an artifact of comparing endpoints at different training-loss levels.

A summary of the combined evidence appears in Section 5.8. We used the same five matched seeds throughout this part, so any differences in the results are not driven by initialization or minibatch order.

5.2 Distance from Initialization

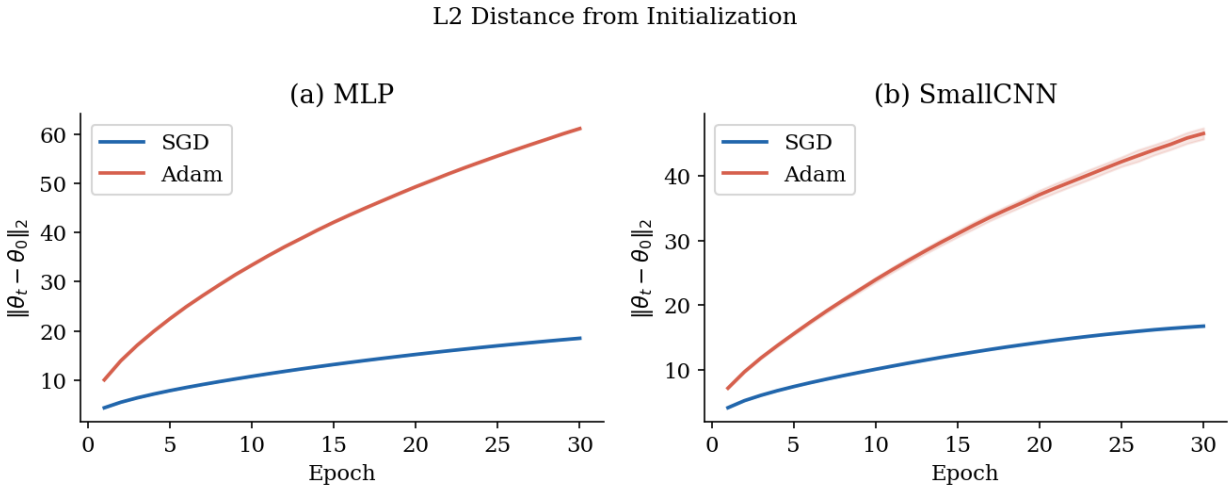


Figure 1: L2 distance from initialization $\|\theta_t - \theta_0\|_2$ over training epochs. Shaded bands show ± 1 standard deviation across five matched seeds. Adam moves substantially farther from the starting point in both architectures; its trajectory is concave (fast early, decelerating later) while SGD drifts roughly linearly.

Figure 1 shows how far each optimizer moves from the shared initialization θ_0. On the MLP, Adam reaches a final parameter distance of 61.20 ± 0.15 versus SGD's 18.49 ± 0.03, a factor of $3.3\times$. On the SmallCNN, the same comparison gives 46.79 ± 0.94 for Adam against 16.75 ± 0.09 for SGD, a factor of $2.8\times$. Both gaps are well outside seed-to-seed variation.

The trajectory shapes are also clearly different. Adam's curve is concave: it advances quickly in the first few epochs and then decelerates, while SGD drifts roughly linearly throughout. This is consistent with Adam's adaptive preconditioning,

$$\eta_{t,i}^{\text{Adam}} = \frac{\alpha}{\sqrt{\hat{v}_{t,i}} + \varepsilon},$$

which evolves as the gradient statistics change during training. We shouldn't read the curve as proof of a single mechanism, but it does show that Adam and SGD aren't simply following the same path at different speeds. Since both runs share θ_0 and the same minibatch order, the most natural explanation for the distance gap is the difference in update rule itself.

That said, this is a parameter-space statement, not a function-space one: neural networks can represent similar functions with very different parameter vectors, especially given scaling symmetries and BatchNorm. So distance from initialization shows that the optimizer trajectories and the selected parameter regions differ, but it does not by itself prove that the input–output functions differ by the same amount. We come back to that question in Section 6.4.

Why not raw gradient norm. We do not use raw gradient norm as evidence of flatness, because Adam updates a preconditioned gradient rather than the raw gradient. The relevant trajectory diagnostics are actual parameter movement: net distance here, and cumulative path length plus per-step update norm in Section 5.3.

5.3 Trajectory Decomposition: Path Length and Update Norm

The previous subsection shows Adam ends much farther from θ_0 than SGD does, but net displacement only describes the endpoint. Two trajectories can land at the same place after very different routes. To distinguish "Adam moves farther because it takes larger steps" from "Adam follows a more directed drift," we now record the cumulative parameter movement and the per-step update norm.

Cumulative path length. For a training run with T minibatch updates, define

$$\mathcal{P}_T = \sum_{t=0}^{T-1} \|\theta_{t+1} - \theta_t\|_2.$$

By the triangle inequality $\mathcal{P}_T \geq \|\theta_T - \theta_0\|_2$. The *directness ratio*

$$R_T = \frac{\|\theta_T - \theta_0\|_2}{\mathcal{P}_T} \in (0, 1]$$

measures how close the trajectory is to a straight-line drift from θ_0 to θ_T. A value near 1 means most of the per-step movement contributes to the net displacement; a small value means most of it cancels out, so the optimizer is moving a lot without getting much farther from where it started.

Update-norm profile. The per-step update norm $\|\Delta\theta_t\|_2 = \|\theta_{t+1} - \theta_t\|_2$ is a stronger signal than the raw gradient norm because it measures the actual change in parameters, including the effect of the optimizer's preconditioner.

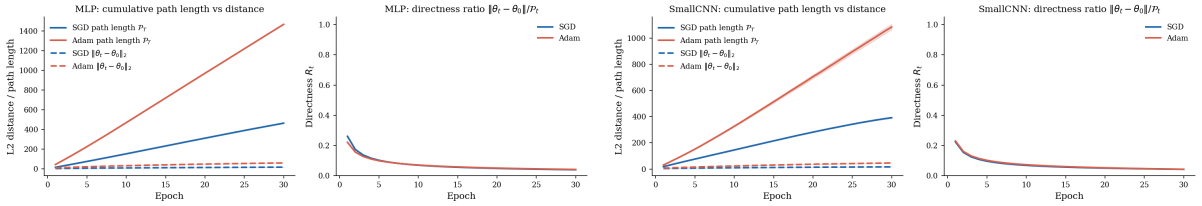
Results. Table 3 summarizes path length, directness ratio, and mean update norm over five matched seeds.

Model	**Opt.**	$\|\theta_T - \theta_0\|_2$	$\mathcal{P}_T$	R_T	$\overline{\|\Delta\theta\|}$
MLP	SGD	18.48 ± 0.04	464.4 ± 1.2	0.0398 ± 0.0001	0.0330 ± 0.0001
MLP	Adam	61.29 ± 0.12	1465.8 ± 1.4	0.0418 ± 0.0001	0.1042 ± 0.0001
SmallCNN	SGD	16.79 ± 0.09	392.0 ± 3.6	0.0428 ± 0.0002	0.0279 ± 0.0003
SmallCNN	Adam	46.62 ± 0.96	1083.8 ± 21.4	0.0430 ± 0.0001	0.0770 ± 0.0015

Table 3: Trajectory geometry: net displacement, cumulative path length $\mathcal{P}_T$, directness ratio R_T, and mean per-step update norm $\overline{\|\Delta\theta\|}$ across five matched seeds.

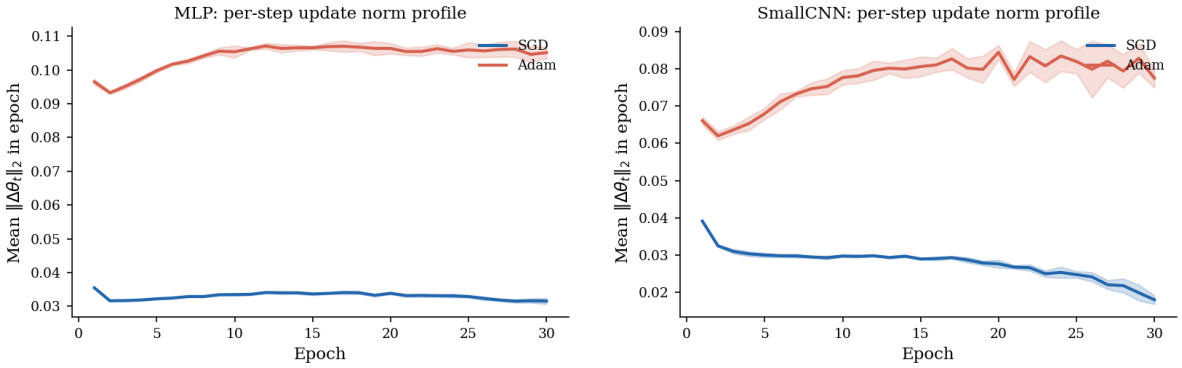
The data resolve the path-vs-step question cleanly. In both architectures, Adam's cumulative path length is roughly 2.8–3.2× that of SGD (1465.8 vs. 464.4 on MLP; 1083.8 vs. 392.0 on SmallCNN), and the mean per-step update norm is 2.8–3.2× larger (0.1042 vs. 0.0330; 0.0770 vs. 0.0279). The directness ratios are essentially identical between the two optimizers (0.0398 vs. 0.0418 on MLP; 0.0428 vs. 0.0430 on SmallCNN), and both are small in absolute terms (~ 0.04): the trajectories are far from straight, but Adam is no more meandering than SGD relative to its own path length.

The factor-three endpoint-distance gap from Section 5.2 is therefore explained by Adam's diagonal preconditioner acting as a coordinate-wise step-size amplifier, not by a qualitatively different geometric strategy. The per-epoch path-length curves (Figure 2) and step-norm profiles (Figure 3) confirm that this scale gap is sustained throughout training, not concentrated in a few early epochs.



(a) MLP (b) SmallCNN

Figure 2: Per-epoch cumulative path length $\mathcal{P}_t$ versus net displacement $\|\theta_t - \theta_0\|_2$. Mean over five matched seeds; bands are ± 1 std. Adam's path length grows much faster than its endpoint distance, so the bigger gap is in step size, not direction.



(a) MLP (b) SmallCNN

Figure 3: Mean per-step update norm $\overline{\|\Delta\theta_t\|}_2$ by epoch, five matched seeds.

5.4 Sharpness and Hessian Trace

We compute two Hessian-based proxies at the final epoch on a fixed mini-batch of 512 examples in evaluation mode.

Sharpness $\lambda_{\max}$ via power iteration. Starting from a random unit vector v_0, we iterate

$$v_{k+1} = \frac{H(\hat{\theta})\, v_k}{\|H(\hat{\theta})\, v_k\|_2}, \qquad \lambda_{\max} \approx v_k^\top H(\hat{\theta})\, v_k,$$

stopping when $|\lambda_k - \lambda_{k-1}| < 10^{-4}$ or after 100 iterations. The Hessian-vector product $H(\hat{\theta})v$ is computed without materializing H using second-order automatic differentiation (Pearlmutter's trick):

$$H(\hat{\theta})\, v = \nabla_\theta \Big[\nabla_\theta \widehat{L}(\hat{\theta})^\top v \Big].$$

Hessian trace $\mathrm{tr}(H)$ via Hutchinson's estimator. For $m = 30$ independent Rademacher vectors $z^{(i)} \sim \{\pm 1\}^d$,

$$\mathrm{tr}(H) = \mathbb{E}\big[z^\top H z\big] \approx \frac{1}{m}\sum_{i=1}^{m} \big(z^{(i)}\big)^\top H(\hat{\theta})\, z^{(i)}.$$

The Hutchinson identity follows from $\mathbb{E}[zz^\top] = I$ for Rademacher vectors:

$$\mathbb{E}[z^\top H z] = \mathbb{E}[\mathrm{tr}(z^\top H z)] = \mathbb{E}[\mathrm{tr}(H z z^\top)] = \mathrm{tr}(H).$$

The two proxies are complementary: $\lambda_{\max}$ identifies the single worst direction, while $\mathrm{tr}(H)$ aggregates curvature across all directions.

Model	**Opt.**	**Test acc.**	**Gap**	$\lambda_{\max}(H)$	$\mathrm{tr}(H)$	**Param dist.**
MLP	SGD	0.8901 ± 0.0024	0.0904 ± 0.0027	36.20 ± 5.97	409.71 ± 30.44	18.49 ± 0.03
MLP	Adam	0.8887 ± 0.0030	0.0912 ± 0.0026	7.22 ± 1.34	68.13 ± 5.04	61.20 ± 0.15
SmallCNN	SGD	0.9166 ± 0.0010	0.0779 ± 0.0028	42.58 ± 15.71	242.47 ± 50.04	16.75 ± 0.09
SmallCNN	Adam	0.9150 ± 0.0009	0.0780 ± 0.0030	64.17 ± 13.88	218.91 ± 61.41	46.79 ± 0.94

Table 4: Final scalar metrics across five matched seeds (mean $\pm$ std). Accuracy and gap are reported as fractions. Gap = train accuracy $-$ test accuracy.

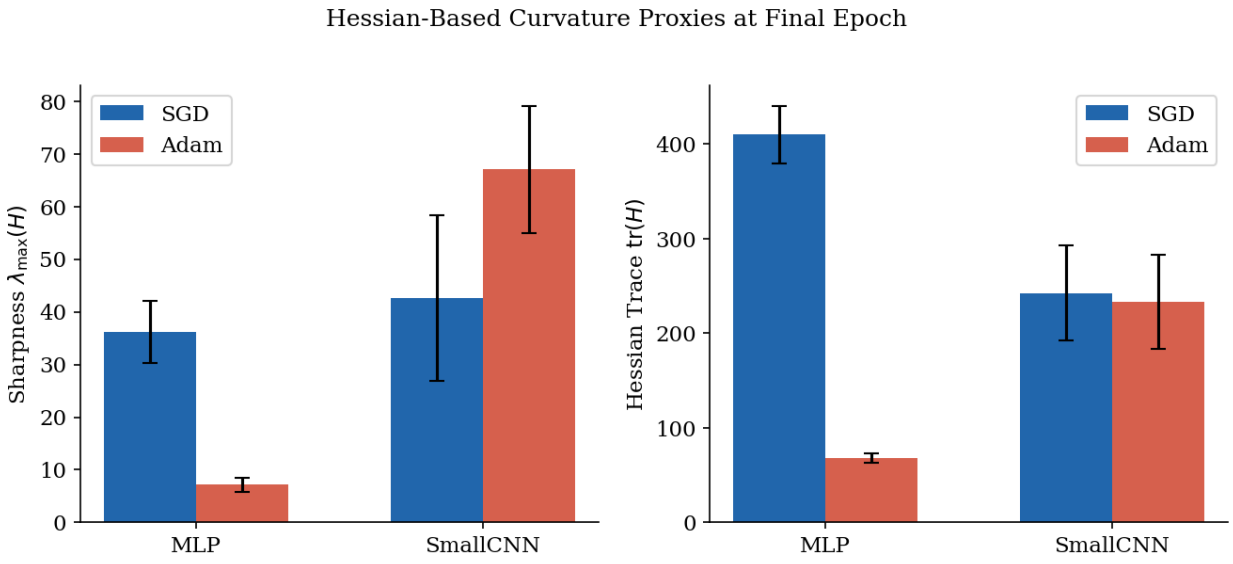


Figure 4: Hessian-based curvature proxies at the final epoch. *Left*: dominant eigenvalue $\lambda_{\max}(H)$. *Right*: Hessian trace $\mathrm{tr}(H)$. Error bars show ± 1 std across five seeds. The MLP shows a clear separation: Adam is substantially flatter on both measures. In the SmallCNN, Adam's $\lambda_{\max}$ is *higher* than SGD's, while the traces are comparable.

MLP. The gap is large and consistent across all five seeds. SGD reaches solutions with $\lambda_{\max} = 36.20 \pm 5.97$ and trace 409.71 ± 30.44, while Adam reaches $\lambda_{\max} = 7.22 \pm 1.34$ and trace 68.13 ± 5.04. Adam's solutions are about $5\times$ flatter in dominant eigenvalue and $6\times$ lower in total curvature; the error bars of the two optimizers do not overlap in either metric.

The MLP result also clarifies the relationship between distance from initialization and sharpness: Adam travels 3.3× farther than SGD, yet ends in a much flatter region. This pattern is plausible geometrically — a long journey can end in a broad valley while a short journey can end near a narrow basin — and the latter description fits SGD: it stays close to θ_0 but arrives in a sharper region.

SmallCNN. The CNN runs in the opposite direction for $\lambda_{\max}$: Adam's mean dominant eigenvalue (64.17 ± 13.88) is higher than SGD's (42.58 ± 15.71), while the Hessian traces are comparable (218.91 vs. 242.47, a difference smaller than one standard deviation). So the CNN case cannot be summarized as simply "Adam is flatter" or "SGD is flatter" — the two metrics disagree. The convolutional inductive bias from weight sharing and pooling appears to pin the *aggregate* curvature $\mathrm{tr}(H)$ to a similar level for both optimizers, even when the worst-direction curvature $\lambda_{\max}$ separates them.

5.5 Perturbation Flatness

The Hessian-based proxies in Section 5.4 are local quadratic measurements. To test whether they reflect actual loss robustness, we now evaluate the loss change under explicit parameter perturbations. The local quadratic approximation predicts

$$\widehat{L}(\theta^\star + \delta) - \widehat{L}(\theta^\star) \approx \frac{1}{2}\,\delta^\top H(\theta^\star)\,\delta,$$

and for isotropic $\delta \sim \mathcal{N}(0, \sigma^2 I)$,

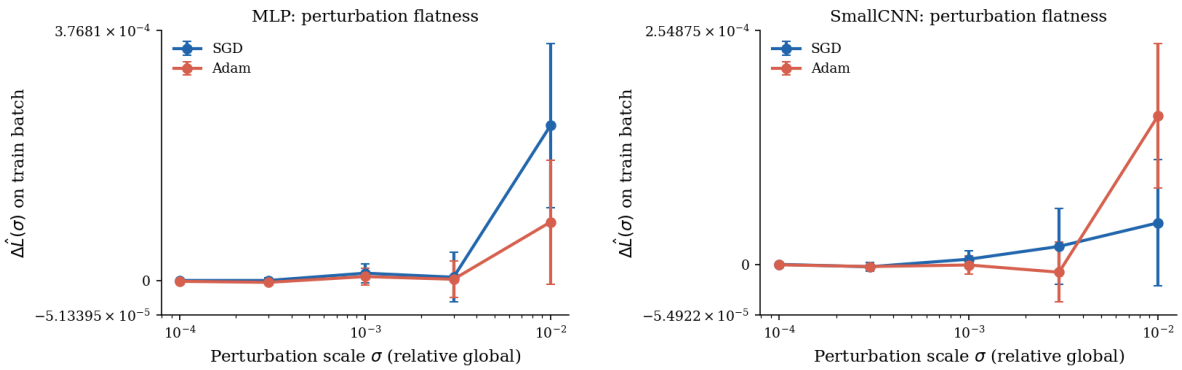
$$\mathbb{E}_\delta\big[\widehat{L}(\theta^\star + \delta) - \widehat{L}(\theta^\star)\big] \approx \tfrac{\sigma^2}{2}\,\mathrm{tr}(H).$$

Relative global perturbation. To make perturbation magnitudes comparable across optimizers whose absolute parameter scales differ (Adam typically reaches points with larger $\|\theta\|_2$), we use a *relative* global perturbation:

$$\delta = \sigma\,\frac{\|\theta^\star\|_2}{\|\xi\|_2}\,\xi, \qquad \xi \sim \mathcal{N}(0, I_d).$$

Equivalently, $\|\delta\|_2 = \sigma\,\|\theta^\star\|_2$, so σ controls the ratio of perturbation norm to parameter norm. We sweep $\sigma \in \{10^{-4}, 3\times10^{-4}, 10^{-3}, 3\times10^{-3}, 10^{-2}\}$ and average over $K = 10$ independent perturbations per σ. Loss is evaluated on a fixed subset of 4096 training examples in evaluation mode (BatchNorm running stats from training).

Results. Figure 5 shows $\Delta\widehat{L}(\sigma)$ versus σ on log–log axes for both architectures and both optimizers.



(a) MLP

(b) SmallCNN

Figure 5: Mean training-loss increase $\Delta\widehat{L}(\sigma)$ under relative isotropic Gaussian perturbations, averaged over five matched seeds and $K = 10$ perturbations per σ. Values at $\sigma \leq 3 \times 10^{-3}$ are at the noise floor ($\leq 10^{-5}$); the operational comparison is at $\sigma = 10^{-2}$.

For $\sigma \in \{10^{-4}, 3\times10^{-4}, 10^{-3}, 3\times10^{-3}\}$, $\Delta\widehat{L}$ sits at $\leq 10^{-5}$ for all four (architecture, optimizer) configurations and is statistically indistinguishable from zero given seed variability — those points just establish a noise floor. The operational comparison is therefore at $\sigma = 10^{-2}$. On the MLP, $\Delta\widehat{L} = (2.3 \pm 1.2) \times 10^{-4}$ for SGD versus $(0.9 \pm 0.9) \times 10^{-4}$ for Adam, a $\sim 2.5\times$ gap in the direction predicted by the trace ratio in Section 5.4, with seed standard deviations well separated from zero for both optimizers. On the SmallCNN, the values are $(0.5 \pm 0.7) \times 10^{-4}$ for SGD and $(1.6 \pm 0.8) \times 10^{-4}$ for Adam: the central tendency goes the same way as the Hessian and loss-matched $\lambda_{\max}$ readings, but both standard deviations overlap zero, so this is a directional observation from $N = 5$ seeds rather than a statistically significant reversal. The perturbation evidence therefore corroborates the Hessian readings only at the operational $\sigma = 10^{-2}$ contrast, and only on the MLP at one-sigma confidence.

5.6 Inter-Seed Parameter Dispersion

We measure each optimizer's reproducibility by the mean pairwise L2 distance over all $\binom{5}{2} = 10$ pairs of final parameter vectors:

$$\bar{d} = \frac{1}{\binom{5}{2}} \sum_{i<j} \|\hat{\theta}_i^* - \hat{\theta}_j^*\|_2.$$

Architecture	SGD mean pairwise dist.	Adam mean pairwise dist.
MLP	31.50	87.72
SmallCNN	27.04	67.33

Table 5: Inter-seed parameter dispersion. Adam produces more dispersed final parameter vectors across seeds in both architectures.

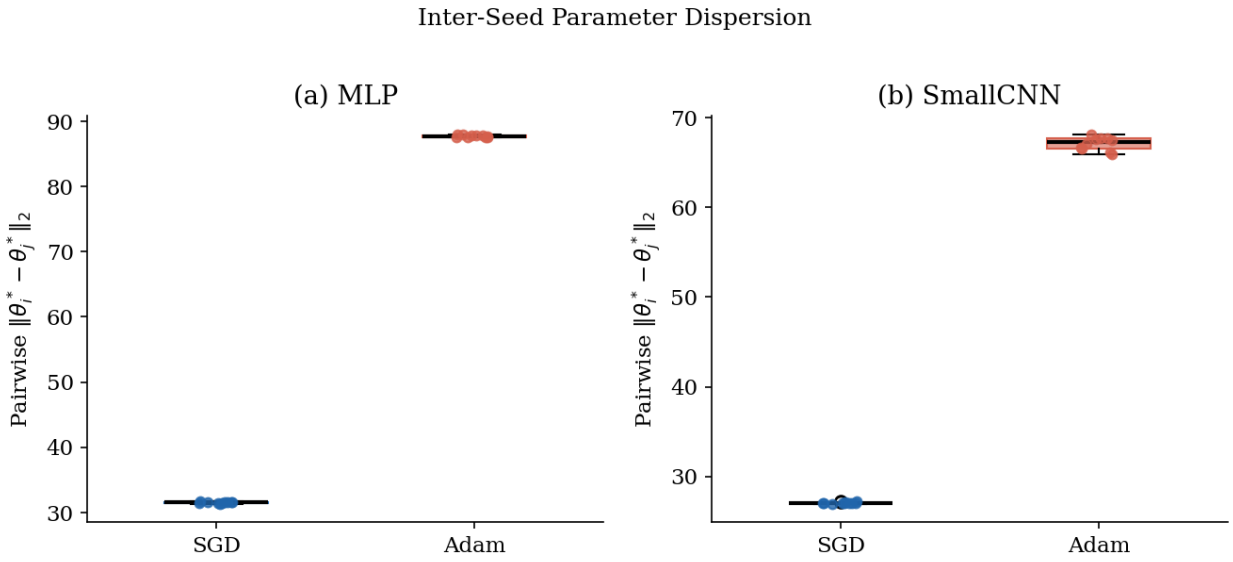


Figure 6: Inter-seed parameter dispersion. Each point is the L2 distance $\|\hat{\theta}_i^* - \hat{\theta}_j^*\|_2$ between final parameter vectors of two seeds, for a fixed optimizer. SGD points are nearly superimposed in both architectures. Adam solutions are about 2.8× more dispersed in the MLP and 2.5× more dispersed in the SmallCNN.

Table 5 and Figure 6 show a consistent pattern across both architectures. SGD solutions are tightly clustered across seeds (MLP: 31.50; CNN: 27.04), with the dots in Figure 6 nearly superimposed. Adam solutions are systematically more dispersed: 87.72 on the MLP (2.8× SGD) and 67.33 on the CNN (2.5× SGD). Within each optimizer the pairwise distances are consistent; the salient signal is that Adam's distances are systematically much larger than SGD's, in both architectures.

Crucially, this dispersion shows up even on the SmallCNN, where the curvature evidence is inconclusive. The optimizer effect is therefore not limited to the MLP: Adam is more sensitive to the stochastic trajectory and lands in more separated regions across seeds. One plausible explanation is Adam's coordinate-wise rescaling — small early differences across seeds can affect different coordinates differently, and adaptive normalization lets those differences accumulate over training.

5.7 Loss-Matched Geometry Control

Motivation. Fixed-epoch comparisons can confound geometric differences with differences in training-loss level. If at epoch 30 Adam reaches a smaller training loss than SGD does, then the Hessian and distance values being compared are taken at different points along each optimizer's loss-versus-time curve. A reader could legitimately ask whether the geometric gap is just an artifact of the loss gap. The loss-matched control answers this question and is the most important falsification test for the geometry verdict that the previous five subsections build up.

Method. For each matched seed and architecture, let $\widehat{L}_{\text{Adam}}^{\text{final}}$ be the final training loss of Adam at epoch T. We then search the SGD trajectory for the epoch

$$t^\star = \arg\min_{t \in \{1,\ldots,T\}} \left| \widehat{L}_{\text{SGD}}(t) - \widehat{L}_{\text{Adam}}^{\text{final}} \right|,$$

load SGD's parameter checkpoint at epoch $t^\star$, and compare its geometry to Adam's final solution. The compared quantities are the same as before: distance from initialization, dominant Hessian

eigenvalue $\lambda_{\max}(H)$, and Hessian trace.

Results. Table 6 reports the loss-matched comparison. The mean matched SGD epoch is $t^\star = 29.8$ for the MLP and $t^\star = 28.8$ for the SmallCNN, indicating that SGD reaches Adam's final loss late in its 30-epoch trajectory in both architectures.

Model	Endpoint	$\|\theta - \theta_0\|_2$	$\lambda_{\max}(H)$	$\mathrm{tr}(H)$
MLP	SGD @ epoch $t^\star$	18.42 ± 0.10	37.15 ± 11.65	437.2 ± 24.3
MLP	Adam @ epoch T	61.29 ± 0.12	6.86 ± 0.79	68.4 ± 9.4
SmallCNN	SGD @ epoch $t^\star$	16.58 ± 0.24	6.17 ± 12.34	221.8 ± 64.9
SmallCNN	Adam @ epoch T	46.62 ± 0.96	61.77 ± 26.43	184.8 ± 57.0

Table 6: Loss-matched control: SGD parameters at the epoch with closest training loss to Adam's final loss, paired with Adam's final parameters. Mean ± std across five matched seeds.

Estimator variance. The standard deviations on $\lambda_{\max}(H)$ in Table 6 are large relative to the means, particularly for the SmallCNN/SGD entry (6.17 ± 12.34). Two sources of noise compound here: power-iteration estimates on a $\sim 10^5$-parameter Hessian are sensitive to the random initialization of the iterate, and $\lambda_{\max}$ itself varies across the five matched seeds. We report mean ± std for comparability with the rest of Part II rather than median/IQR. Despite this, the MLP comparison (37.15 vs. 6.86) is statistically distinguishable at one sigma, and the SmallCNN reversal (6.17 vs. 61.77) should be read as a directional finding from a single architecture pair rather than a tight quantitative claim.

The architecture comparison is striking. On the MLP, Adam's $\lambda_{\max}$ is roughly 5.4× smaller than SGD's at matched loss (6.86 vs. 37.15), and the trace is 6.4× smaller (68.4 vs. 437.2): the flatness gap from Section 5.4 survives the loss-matching control intact, so the two optimizers genuinely select different curvature regimes and Adam's flatter MLP landscape is not just a side effect of having reached lower training loss. On the SmallCNN, the picture inverts: Adam's $\lambda_{\max}$ is roughly 10× *larger* than SGD's at matched loss (61.77 vs. 6.17), while the trace is comparable (184.8 vs. 221.8). The opposite sign on the SmallCNN, even after loss matching, says the optimizer-induced flatness ordering is not a universal property of Adam-vs-SGD — at least one of these architecture-optimizer combinations produces a different selection. Given the single architecture pair and the high variance of $\lambda_{\max}$ noted above, we read this as a directional finding that motivates replication on a larger architecture sample rather than as an established rule.

Function-space disagreement also persists at matched loss. At $t^\star$ versus T, D_{pred} is 0.0772 ± 0.0066 on the MLP and 0.0692 ± 0.0024 on the SmallCNN, while symmetric KL is 0.1880 and 0.2858 respectively—values comparable to the standard final-epoch comparison reported in Section 6.4. Even after controlling for training progress, SGD and Adam have selected different predictors.

5.8 Part II Synthesis

The six measurements line up into a consistent geometry verdict. Adam ends much farther from initialization in both architectures, and the trajectory decomposition pins this on bigger per-step updates rather than a more directed walk. On the MLP, the Hessian proxies, the perturbation evidence at the operational σ, and the loss-matched control all agree that Adam picks markedly flatter solutions. On the SmallCNN the picture is more mixed: Hessian and loss-matched $\lambda_{\max}$ flip the sign, and perturbation flatness only gives a directional read. Inter-seed dispersion, on the other

hand, shows the Adam-vs-SGD signal in both architectures, so the optimizer effect isn't an MLP-only artifact. And the loss-matched control rules out the simplest alternative explanation: even at the same training loss, the MLP flatness gap and the cross-optimizer function-space disagreement are both still there.

The big caveat is that this Part II story does *not* hand Adam a generalization win on the MLP. Test accuracy is marginally lower (88.87% vs. 89.01%) and train–test gap marginally larger (9.12% vs. 9.04%), and both gaps are inside seed-to-seed noise. That's the tension Part III takes up: Part II shows the geometry really does differ, but does any of it actually reach the predictions?

6 Part III: Generalization and the Propagation of Implicit Bias

6.1 From Geometry to Generalization: Why More Levels Are Needed

Part II shows Adam and SGD pick geometrically distinct parameter regions, and the loss-matched control says this is not just "Adam got to low loss earlier." So the third question is whether any of that matters for generalization. The straightforward answer — final test accuracy and train–test gap — is negative at this protocol scale, and on the MLP it goes in the direction *opposite* to a simple flat-minima story: Adam reaches solutions 5× flatter in $\lambda_{\max}$, yet does slightly worse on test accuracy.

That mismatch is the central puzzle of this project. If the parameter-space difference is real but doesn't show up at test time, then somewhere along the chain "parameters → basin → predictor → representation → test loss" the optimizer signal has to be compressing. We therefore step through three further levels of geometry, each one further from raw parameter values:

1. Basin geometry (Section 6.3): are the two endpoints in the same low-loss basin, or in different ones?

2. Function-space similarity (Section 6.4): do the two endpoints realize the same predictor on test inputs?

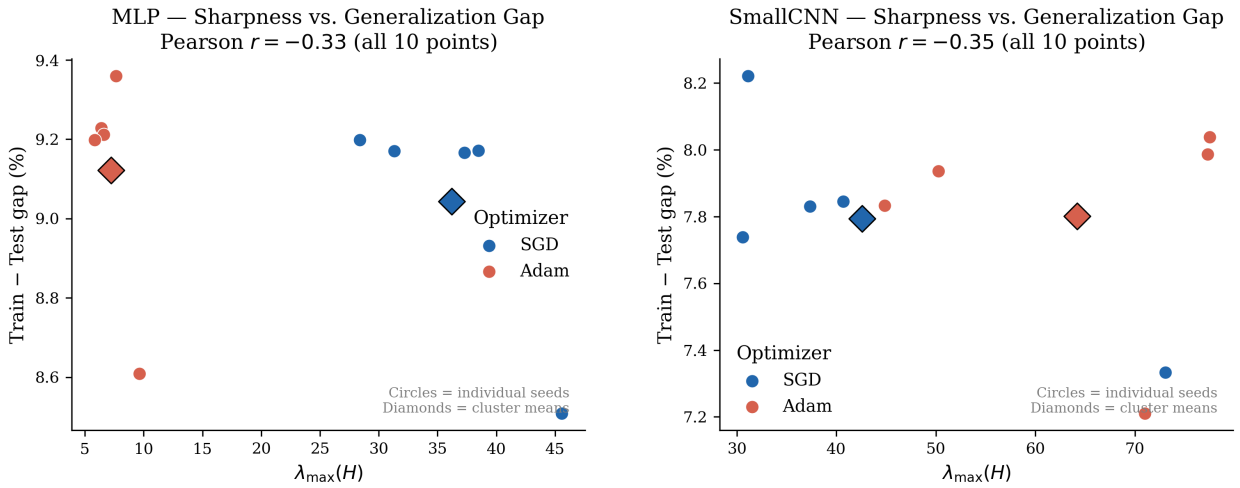
3. Representation-space alignment (Section 6.5): do they share penultimate-layer features, after the predictor head is removed?

At every level we add a same-optimizer cross-seed baseline (10 seed pairs per architecture, per optimizer), so "optimizer matters" is always tested against the reference scale of "seed matters." Section 6.6 pulls the verdicts together. This part is not optional: it is what makes the Part II / Part III mismatch interpretable.

6.2 Test Accuracy and Sharpness–Gap Relationship

Table 7: Final scalar generalization metrics across five matched seeds (mean ± std). Gap = train accuracy − test accuracy.

Model	Opt.	Test acc. (%)	Gap (%)	$\lambda_{\max}(H)$
MLP	SGD	89.01 ± 0.24	9.04 ± 0.27	36.20 ± 5.97
MLP	Adam	88.87 ± 0.30	9.12 ± 0.26	7.22 ± 1.34
SmallCNN	SGD	91.66 ± 0.10	7.79 ± 0.28	42.58 ± 15.71
SmallCNN	Adam	91.50 ± 0.09	7.80 ± 0.30	64.17 ± 13.88



(a) MLP

(b) SmallCNN

Figure 7: Final-epoch sharpness $\lambda_{\max}(H)$ versus train–test gap. Each marker is one matched-seed run; diamonds are cluster centroids. In the MLP, the optimizer clusters are well-separated in sharpness ($\lambda_{\max} \approx 36$ for SGD vs. ≈ 7 for Adam) but not in gap. In the SmallCNN, the optimizer clusters overlap on both axes.

MLP. Adam reaches solutions with $\lambda_{\max} = 7.22 \pm 1.34$ against SGD's 36.20 ± 5.97, a $5\times$ flatness gap. A simple flat-minima account [2, 3] would predict this to show up as better generalization. The data point the other way: SGD has marginally higher test accuracy (89.01% vs. 88.87%) and a marginally smaller train–test gap (9.04% vs. 9.12%). Both differences are inside the seed-to-seed standard deviation, so we don't claim a real ranking — but the *direction* is striking, since lower Euclidean Hessian sharpness here doesn't buy any generalization benefit. Figure 7 makes this visual: the two optimizer clusters are wide apart on the sharpness axis but vertically overlapping on the gap axis.

SmallCNN. Generalization is essentially tied (91.66% vs. 91.50%; gap 7.79% vs. 7.80%), and the Hessian ordering is mixed — Adam has higher $\lambda_{\max}$ but a comparable trace. The convolutional inductive bias seems to push both optimizers toward similarly performing solutions, even when their parameter trajectories don't agree.

So the simple flat-minima story isn't enough at this protocol scale. Whatever determines generalization here has to come through architecture, basin connectivity, function-space behavior, and representation similarity. The next three subsections take each one in turn.

6.3 Basin Geometry: Linear Mode Connectivity

Motivation. Part II tells us that Adam's solutions are far from SGD's in Euclidean parameter space. The natural follow-up is whether they actually sit in different basins, or just in different parts of the same basin. Linear mode connectivity [6, 7] probes this directly: if the straight line between two minima never rises above the larger of their two losses, they share a low-loss region. So this is the basin-level version of the distance result in Section 5.2 — it turns "far apart in parameter space" into a yes/no question about connectivity.

Method. For each matched seed and architecture, define the line

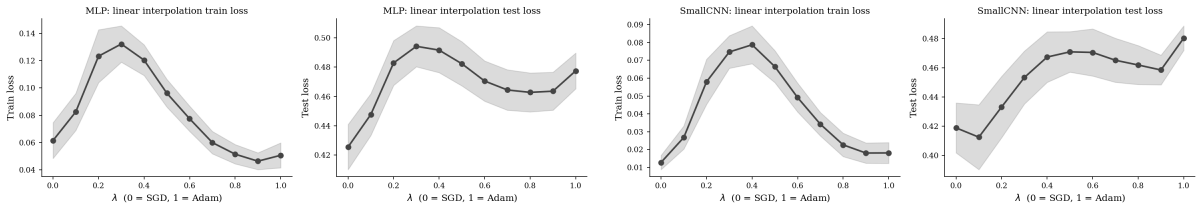
$$\theta(\lambda) = (1-\lambda)\,\theta_{\text{SGD}} + \lambda\,\theta_{\text{Adam}}, \qquad \lambda \in \{0, 0.1, 0.2, \ldots, 1.0\}.$$

We evaluate training and test loss at each λ. For BatchNorm models, the running statistics interpolate to values that no longer match the data, so we recalibrate them at each interior λ by a single forward pass over 50 training batches in train mode (no parameter update). The endpoints $\lambda = 0$ and $\lambda = 1$ keep their original BN stats. The loss *barrier* is

$$B = \max_{\lambda \in [0,1]} \widehat{L}(\theta(\lambda)) - \max\left\{\widehat{L}(\theta_{\text{SGD}}),\ \widehat{L}(\theta_{\text{Adam}})\right\}.$$

$B \approx 0$ indicates linear connectivity; $B > 0$ indicates a barrier.

Results. Figure 8 plots $\widehat{L}(\theta(\lambda))$ over the interpolation, and Table 8 reports B averaged over seeds.



(a) MLP (b) SmallCNN

Figure 8: Linear mode connectivity: training and test loss along $\theta(\lambda) = (1-\lambda)\theta_{\text{SGD}} + \lambda\theta_{\text{Adam}}$, with BatchNorm recalibrated at interior λ. Solid line is mean over five matched seeds; band is ± 1 std.

Model	**Comparison**	**Train barrier B_{train}**	**Test barrier B_{test}**
MLP	SGD-vs-Adam (matched seed)	0.0721 ± 0.0086	0.0174 ± 0.0086
MLP	SGD-vs-SGD (cross seed)	0.4952 ± 0.0630	0.2855 ± 0.0614
MLP	Adam-vs-Adam (cross seed)	0.4925 ± 0.0401	0.2528 ± 0.0389
SmallCNN	SGD-vs-Adam (matched seed)	0.0615 ± 0.0078	0.0032 ± 0.0065
SmallCNN	SGD-vs-SGD (cross seed)	1.1843 ± 0.2540	0.8102 ± 0.2547
SmallCNN	Adam-vs-Adam (cross seed)	1.4214 ± 0.2918	0.9975 ± 0.2918

Table 8: Loss barrier along the linear interpolation between two trained solutions. Top sub-block per architecture: matched-seed SGD-vs-Adam (mean $\pm$ std over five seeds). Lower sub-blocks: same-optimizer cross-seed baseline averaged over $C(5,2) = 10$ seed pairs. The matched-seed cross-optimizer barrier is roughly an order of magnitude smaller than either same-optimizer cross-seed baseline.

The matched-seed cross-optimizer train-loss barriers are small but consistently positive (0.0721 on the MLP, 0.0615 on the SmallCNN), and well outside the cross-seed standard deviations. So there really is a reproducible bump in training loss along the straight line between matched SGD and Adam, even after BatchNorm recalibration — the two endpoints aren't in the exact same basin. But the same-optimizer cross-seed baseline puts that bump in perspective: $B^{\text{SS}}_{\text{train}} = 0.4952 \pm 0.0630$ and $B^{\text{AA}}_{\text{train}} = 0.4925 \pm 0.0401$ on the MLP, 1.1843 ± 0.2540 / 1.4214 ± 0.2918 on the SmallCNN. That makes the matched-seed cross-optimizer barrier roughly $7\times$ smaller than the same-optimizer

cross-seed barrier on the MLP, and $\sim 20\times$ smaller on the SmallCNN. Two SGD runs from different seeds end up basin-separated on a much larger scale than matched SGD and Adam do.

This lines up with Frankle et al. [7]: under standard training, two same-optimizer runs from different initializations sit in linearly disconnected basins, and our SGD-vs-SGD / Adam-vs-Adam baselines reproduce that effect at our scale. So what drives basin connectivity in this protocol is mostly the shared seed (init plus minibatch order), not which optimizer was used. The cross-optimizer effect is a small extra bump on top of an already highly connected matched-seed landscape. Test-loss barriers tell the same story on a smaller scale: 0.0174 / 0.0032 for matched-seed cross-optimizer pairs against ~ 0.25–1.0 for same-optimizer cross-seed pairs.

6.4 Function-Space Similarity

Motivation. Parameter-space distance doesn't directly tell us how different two trained networks are *as functions*. Two networks can have wildly different weights and still make nearly identical predictions, especially with scale symmetries and BatchNorm in play. So we now look at three function-level metrics on the test set — the predictor-level analogue of Part II's measurements, asking whether the parameter difference is still there once we look at outputs.

Metrics. For matched SGD and Adam models with predicted class probabilities $p_S(\cdot|x)$ and $p_A(\cdot|x)$ and pre-softmax logits $z_S(x), z_A(x)$, we compute, over the N-point test set:

$$D_{\text{pred}} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\big[\arg\max_k p_S(k|x_i) \neq \arg\max_k p_A(k|x_i)\big], \tag{1}$$

$$D_{\text{SKL}} = \frac{1}{2N}\sum_{i=1}^{N} \big[D_{\text{KL}}(p_S\|p_A) + D_{\text{KL}}(p_A\|p_S)\big], \tag{2}$$

$$C_{\text{logit}} = \frac{1}{N}\sum_{i=1}^{N} \frac{z_S(x_i)^\top z_A(x_i)}{\|z_S(x_i)\|_2\,\|z_A(x_i)\|_2}. \tag{3}$$

D_{pred} is a strict decision-level disagreement; D_{SKL} measures the divergence of the full softmax distribution; C_{logit} is scale-invariant and captures direction similarity in logit space.

Results. Table 9 reports the three metrics.

Model	Comparison	D_{pred}	D_{SKL}	C_{logit}
MLP	SGD-vs-Adam (matched seed)	0.0738 ± 0.0028	0.1749 ± 0.0051	0.6122 ± 0.0058
MLP	SGD-vs-SGD (cross seed)	0.0868 ± 0.0067	0.2056 ± 0.0286	0.9187 ± 0.0035
MLP	Adam-vs-Adam (cross seed)	0.0822 ± 0.0042	0.2348 ± 0.0195	0.9561 ± 0.0022
SmallCNN	SGD-vs-Adam (matched seed)	0.0671 ± 0.0042	0.2676 ± 0.0219	0.6516 ± 0.0061
SmallCNN	SGD-vs-SGD (cross seed)	0.0740 ± 0.0045	0.3121 ± 0.0324	0.9050 ± 0.0039
SmallCNN	Adam-vs-Adam (cross seed)	0.0786 ± 0.0031	0.4038 ± 0.0212	0.9295 ± 0.0040

Table 9: Function-space similarity on the Fashion-MNIST test set. Top sub-block per architecture: matched-seed SGD-vs-Adam (mean ± std over five seeds). Lower sub-blocks: same-optimizer cross-seed baseline averaged over $C(5,2) = 10$ pairs. D_{pred} and D_{SKL} are at or below the same-optimizer cross-seed baseline; only the logit-direction cosine C_{logit} shows a sharp matched-seed cross-optimizer signal.

The two matched-seed predictors disagree on 7.4% of MLP test inputs and 6.7% on the SmallCNN. The same-optimizer cross-seed baselines for D_{pred} are 0.0868 ± 0.0067 (SGD-vs-SGD) and 0.0822 ± 0.0042 (Adam-vs-Adam) on the MLP, and 0.0740 ± 0.0045 / 0.0786 ± 0.0031 on the SmallCNN. So matched-seed cross-optimizer D_{pred} is at or slightly *below* the same-optimizer cross-seed baseline. Symmetric KL behaves the same way (0.1749 for cross-optimizer matched seed on the MLP, against 0.20–0.23 for same-optimizer cross-seed). At decision and softmax level, in other words, optimizer choice doesn't push the disagreement above what swapping seeds already does.

The one exception is the logit-direction cosine. Matched SGD-vs-Adam gives $C_{\text{logit}} \approx 0.61$–$0.65$, versus 0.92–0.96 within either optimizer across seeds. Optimizer choice clearly shifts the pre-softmax logit *direction*, but argmax and the full softmax absorb most of that shift before it reaches a prediction. There's also a small architecture effect worth flagging: the SmallCNN has slightly lower D_{pred} but higher D_{SKL} than the MLP, so when the CNN models do disagree they tend to disagree more confidently in distribution.

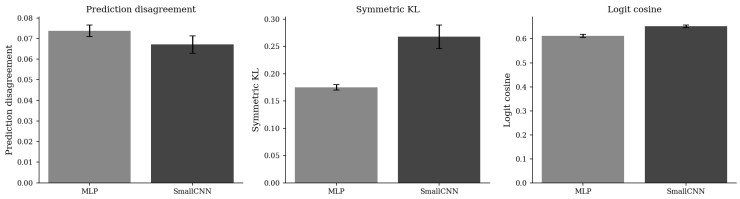


Figure 9: Function-space disagreement metrics on Fashion-MNIST test set, mean ± std over five seeds.

6.5 Representation-Space Alignment

Function-space metrics compare outputs, but two models can produce similar outputs while running on different internal representations. So we compare penultimate-layer features directly using linear centered kernel alignment (CKA) [8]. Let F_S and F_A be centered feature matrices for matched SGD and Adam on the same test subset. Linear CKA is

$$\text{CKA}(F_S, F_A) = \frac{\|F_S^\top F_A\|_F^2}{\|F_S^\top F_S\|_F \, \|F_A^\top F_A\|_F}.$$

CKA is invariant to isotropic rescaling and far more stable than raw feature distance for this kind of comparison. This is the deepest level in the "parameters → basin → predictor → representation" chain: with the classifier head stripped off, do the two optimizers build the same internal feature geometry?

Model	**Comparison**	**Feature kernel alignment A**	**Linear CKA**
MLP	SGD-vs-Adam (matched seed)	0.9697 ± 0.0010	0.8578 ± 0.0059
MLP	SGD-vs-SGD (cross seed)	0.9698 ± 0.0008	0.8613 ± 0.0038
MLP	Adam-vs-Adam (cross seed)	0.9714 ± 0.0008	0.8497 ± 0.0071
SmallCNN	SGD-vs-Adam (matched seed)	0.9664 ± 0.0027	0.9147 ± 0.0113
SmallCNN	SGD-vs-SGD (cross seed)	0.9774 ± 0.0006	0.9220 ± 0.0033
SmallCNN	Adam-vs-Adam (cross seed)	0.9677 ± 0.0024	0.8920 ± 0.0107

Table 10: Representation-space alignment between penultimate-layer features. Matched cross-optimizer values are close to same-optimizer cross-seed baselines, suggesting that optimizer choice does not strongly reshape representation geometry at this protocol scale.

The headline is that representation similarity stays high. Matched SGD-vs-Adam CKA is 0.858 on the MLP and 0.915 on the SmallCNN, both close to the corresponding same-optimizer cross-seed baselines. The large parameter-space separation, in other words, doesn't become a large representation-space separation. Together with the function-space numbers this suggests a layered picture: optimizer choice changes parameter trajectories strongly, logit directions moderately, and penultimate features only weakly. That fits the negative generalization result we opened Section 6 with — the parameter-level signal has already compressed a lot by the time it reaches the levels where test accuracy is actually decided.

6.6 Three-Level Synthesis: Parameter, Basin, Function–Representation

The whole Part II / Part III sequence sits on three levels of geometry, summarized in Table 11.

Level	Quantities	What it answers	Verdict (this study)
Parameter geometry (Part II)	$\|\theta_T - \theta_0\|_2$, $\mathcal{P}_T$, R_T, $\lambda_{\max}(H)$, $\mathrm{tr}(H)$, $\bar{d}_{\text{seed}}$, $\Delta\widehat{L}(\sigma)$	Where does the optimizer go, and what is the local curvature?	Adam path length $\sim 3\times$ SGD's in both architectures; directness ratio essentially equal. MLP: Adam $\sim$ 5–6$\times$ flatter at matched loss. Small-CNN: ordering may reverse, but this is a single-architecture-pair observation with high $\lambda_{\max}$ estimator variance.
Basin geometry (Part III)	Linear-interpolation loss curve, barrier B	Are the two solutions in the same low-loss region?	Cross-optimizer matched-seed train barrier (0.072 MLP, 0.062 CNN) is $\sim 7\times$–$20\times$ smaller than the same-optimizer cross-seed baseline (0.49 MLP, 1.18–1.42 CNN). The matched-seed protocol drives connectivity; cross-optimizer adds a small additional barrier. Test barrier ≤ 0.02 for matched-seed pairs.
Function and representation geometry (Part III)	D_{pred}, D_{SKL}, C_{logit}, $A(K_S, K_A)$, CKA	Do parameter differences correspond to different predictors or representations?	D_{pred} ($\sim$ 7%) and CKA (0.86–0.91) are within the same-optimizer cross-seed baseline range; only $C_{\text{logit}} \approx 0.6$ (vs. 0.92–0.96 within-optimizer) shows a sharp matched-seed cross-optimizer signal. Optimizer choice reshapes logit direction but not argmax decisions or representation geometry beyond seed noise at this scale.

Table 11: Three-level synthesis of the implicit-bias evidence.

Once the same-optimizer cross-seed baseline is in view, the data support a conservative version of the story we expected going in:

Adam genuinely shifts parameter-space trajectory and local Euclidean geometry on the

MLP, and the shift survives loss matching. But as the comparison moves from parameters toward predictions, most of what first looked like a cross-optimizer effect compresses to within the same-optimizer cross-seed baseline: D_{pred}, D_{SKL}, A, and linear CKA on matched-seed cross-optimizer pairs all sit inside the range two runs of one optimizer span across seeds. The matched-seed protocol pushes basin connectivity an order of magnitude beyond what same-optimizer cross-seed pairs reach; on top of that, the cross-optimizer matched-seed train barrier is small but nonzero, and the one function-level metric that shows a sharp cross-optimizer signal above seed noise is the logit-direction cosine C_{logit}. Architecture mediates which curvature regime each optimizer ends up in, but the Small-CNN flatness reversal comes from a single architecture pair with high $\lambda_{\max}$ estimator variance and needs to be replicated.

That is the core message of the project. Optimizer-induced parameter geometry is real, especially on the MLP, and not just a final-loss artifact. But once the same-optimizer cross-seed baseline is included, most of the function- and representation-level difference between matched SGD and Adam at this protocol scale is accounted for by seed-induced variability rather than optimizer-induced bias, with C_{logit} as the main exception.

7 Overall Discussion

The original question was whether adaptive optimization changes implicit bias and generalization. The answer is layered. Adam clearly changes parameter-space trajectory: it moves farther from θ_0, travels a longer path, and produces more dispersed endpoints. On the MLP, that difference also survives the loss-matching control and shows up as much lower Hessian curvature and a smaller perturbation loss increase. On the CNN, the curvature ordering is less stable, which suggests architecture can take over or redirect the optimizer-induced geometry.

But the effect weakens as we step away from parameters. Linear mode connectivity says matched SGD and Adam endpoints are not literally the same point in one basin — the train-loss barrier is small but nonzero — yet that barrier is far smaller than the same-optimizer cross-seed baseline. Function-space metrics show a real logit-direction shift, yet prediction disagreement and symmetric KL stay near the same-optimizer cross-seed baseline. Representation CKA is more compressed still: penultimate features stay highly aligned across optimizers. The shortest version is:

Adam changes geometry strongly in parameter space, mildly in basin and logit geometry, and weakly in representation geometry.

That's why the generalization story isn't a clean win for either side. The MLP shows the largest flatness advantage for Adam, yet Adam doesn't gain a clear edge in test accuracy or gap. The CNN gets similar generalization out of both optimizers even when their parameter trajectories don't agree. The direction of this conclusion lines up with Wilson et al. [5], who report that adaptive methods can generalize worse than well-tuned SGD — but the mechanism in our MLP setting is the *opposite* of the one they invoke. They attribute the gap to Adam picking *sharper* minima; in our matched-seed Fashion-MNIST MLP, Adam picks substantially *flatter* ones and still doesn't generalize better. So optimizer-induced implicit bias is real, but its effect on test performance gets filtered through architecture, loss level, basin connectivity, functional equivalence, and representation similarity before it actually lands.

7.1 Scale-Aware Sharpness and Reparameterization Caveat

The Hessian quantities in this report are Euclidean parameter-space diagnostics. They're useful for controlled within-architecture comparisons, but they aren't invariant measures of function complexity. For a two-layer ReLU network

$$f(x) = W_2\sigma(W_1x),$$

positive homogeneity gives

$$W_2\sigma(W_1x) = (c^{-1}W_2)\sigma(cW_1x), \qquad c > 0 :$$

two parameterizations representing the exact same function can still have different norms and different Hessian spectra. BatchNorm adds related scale freedoms. This reparameterization caveat [4] is exactly why we treat $\lambda_{\max}(H)$, $\mathrm{tr}(H)$, and perturbation flatness as within-protocol diagnostics, not universal generalization measures.

7.2 Limitations

This study uses one dataset, two small architectures, five seeds, and a finite hyperparameter sweep. Hessian estimates are mini-batch proxies, and BatchNorm makes both sharpness and mode-connectivity interpretation a bit subtle. The no-regularization protocol is useful for isolating optimizer effects, but it isn't a production training recipe. The architecture-dependent flatness reversal should therefore be read as a directional observation worth replicating on a wider architecture sample, not as a universal Adam-vs-SGD law.

8 Conclusion

This project compared SGD with momentum and Adam as two preconditioned stochastic dynamics, with the whole investigation built around three linked questions: convergence, solution geometry, and generalization. Mathematically, the contrast is scalar spatial preconditioning for SGD versus a time-varying diagonal preconditioner for Adam, which gives different local error dynamics through

$$e_{t+1} \approx (I - P_tH)e_t.$$

The geometry question is answered with six complementary measurements — distance, path length and update norm, Hessian sharpness/trace, perturbation flatness, inter-seed dispersion, and a loss-matched control. Together they show Adam picks different parameter-space regions under matched seeds, and that this isn't just an artifact of having reached a different training loss. The generalization question then follows that parameter-space difference through three further levels of geometry — basin, function, and representation — and the optimizer signal compresses substantially as it propagates: matched-seed cross-optimizer basin barriers are an order of magnitude smaller than same-optimizer cross-seed barriers, prediction disagreement and CKA fall inside the same-optimizer cross-seed baseline, and only the logit-direction cosine still shows a sharp cross-optimizer signal. So the final lesson isn't that Adam or SGD is universally better. It's that adaptive preconditioning changes which solution gets selected, while generalization depends on how much of that parameter-space change actually survives through architecture, basin structure, function space, and representation geometry.

References

[1] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. International Conference on Learning Representations, 2015. https://arxiv.org/abs/1412.6980.

[2] Sepp Hochreiter and Jürgen Schmidhuber. Flat Minima. Neural Computation, 9(1):1–42, 1997. https://doi.org/10.1162/neco.1997.9.1.1.

[3] Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima. International Conference on Learning Representations, 2017. https://openreview.net/forum?id=H1oyRlYgg.

[4] Laurent Dinh, Razvan Pascanu, Samy Bengio, and Yoshua Bengio. Sharp Minima Can Generalize For Deep Nets. International Conference on Machine Learning, 2017. https://proceedings.mlr.press/v70/dinh17b.html.

[5] Ashia C. Wilson, Rebecca Roelofs, Mitchell Stern, Nati Srebro, and Benjamin Recht. The Marginal Value of Adaptive Gradient Methods in Machine Learning. Advances in Neural Information Processing Systems, 2017. https://proceedings.neurips.cc/paper/2017/hash/81b3833e2504647f9d794f7d7b9bf341-Abstract.html.

[6] Timur Garipov, Pavel Izmailov, Dmitrii Podoprikhin, Dmitry P. Vetrov, and Andrew Gordon Wilson. Loss Surfaces, Mode Connectivity, and Fast Ensembling of DNNs. Advances in Neural Information Processing Systems, 2018. https://proceedings.neurips.cc/paper/2018/hash/be3087e74e9100d4bc4c6268cdbe8456-Abstract.html.

[7] Jonathan Frankle, Gintare Karolina Dziugaite, Daniel M. Roy, and Michael Carbin. Linear Mode Connectivity and the Lottery Ticket Hypothesis. International Conference on Machine Learning, 2020. https://proceedings.mlr.press/v119/frankle20a.html.

[8] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of Neural Network Representations Revisited. International Conference on Machine Learning, 2019. https://proceedings.mlr.press/v97/kornblith19a.html.